

HW2 Multi-Agent Werewolf Prediction

Name: 陳博文 Chen Po-Wen
Student_ID: P14942A08

1. System Design and Architecture

For each queried player in a long, machine-translated, deception-heavy game log I predict a categorical **role** and a continuous **wolf_score**. I designed a **six-stage multi-agent pipeline** that separates *deterministic game-rule extraction* from *LLM behavioural reasoning*, then resolves a legal assignment with a constrained solver and a final deterministic case-audit. The architectural hypothesis I set out to test, and which Section 3 confirms, is that a signal generalises from the 20-game validation set to the 30-game scored set only if its premise is a regex-verifiable game event (an execution or a night-kill); behavioural-statistical tuning on the small validation set does not. The runtime system has six stages: **A1** regex header-parser producing `role_counts`; **A2** evidence-extractor (posts, name resolution, CO, divination, executions, night-deaths); **A3** the only LLM stage (Qwen-7B, three prompt variants, RAG-grounded); **A3.5** Medium-Execution chain; **A4** constrained solver; **A5** deterministic case-audit. I treat each stage as a specialized agent (a parser agent, an evidence agent, three LLM reasoning agents, a solver agent and an audit agent); the LLM agents are the only generative components, while the other agents are deterministic validators or constraint solvers. Figure 1 shows this final inference pipeline; only A3 uses a generative LLM, so failures are easy to localize and ablate one agent at a time.

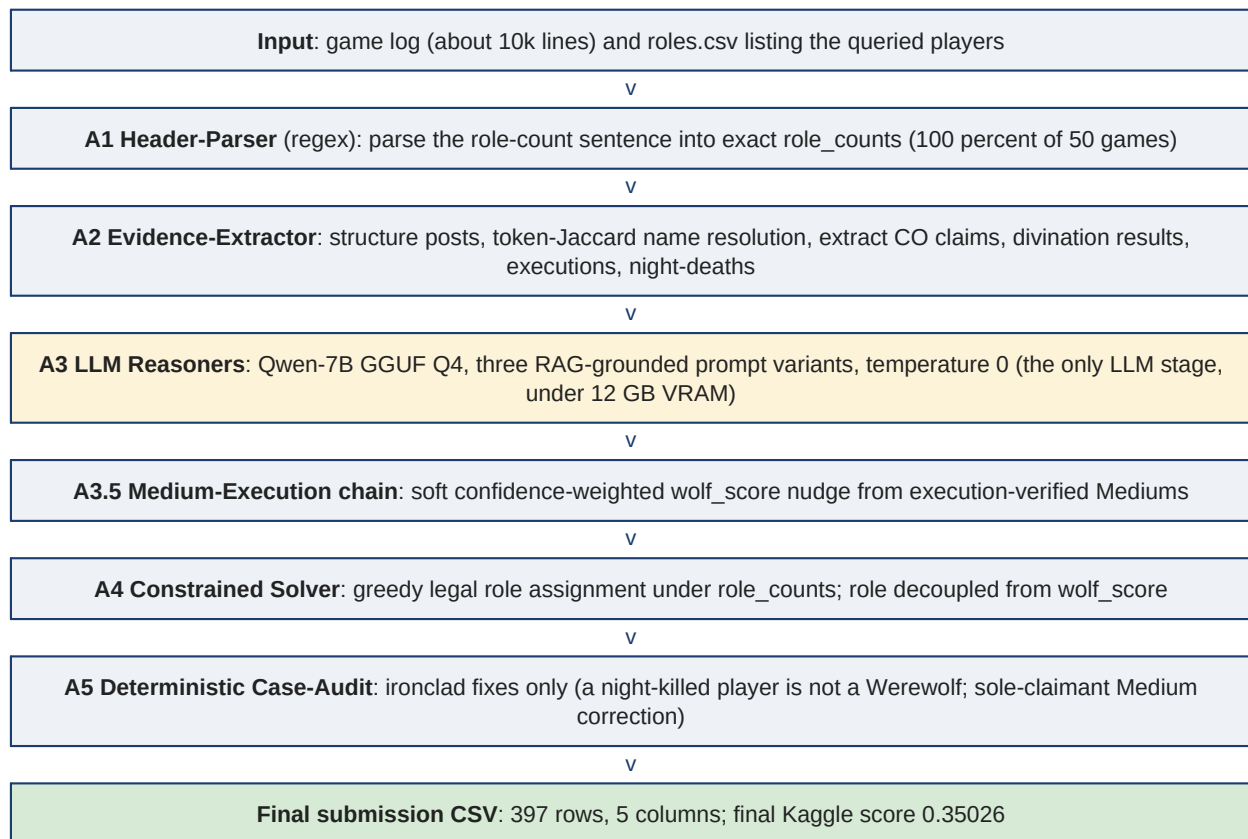


Figure 1. The final inference pipeline that produced the submission (data flows top to bottom). Gold marks the only LLM stage; green the final output; all other stages are deterministic.

1.1 RAG System Design

Posts are embedded with `all-mpnet-base-v2` (local, about 0.4 GB) and cached per game (50 games, a one-off 67 seconds). The query is one player; the retriever is a **hybrid union** of the player's own posts, alias-aware mentions of the player, the reply chains around those, and embedding top-k of the player's concatenated speech, de-duplicated and truncated recency-first to about 12 posts. `mpnet` was

chosen over MiniLM because paraphrase-heavy translation needs sentence-level fidelity more than speed; the 12-post cap is the empirical precision and recall knee (below 8 still missed a late CO; above 18 induced hallucination). Retrieved posts plus the compact A2 evidence JSON form the single A3 context, so input is bounded regardless of game length.

1.2 Issues Faced and How I Solved Them

(1) Context overflow (about 10k lines exceed the 7B window, truncation lost late reveals): feed only the 12-post union plus evidence JSON. **(2) Embedding top-k missed the decisive post** (favoured similar small talk): use the structural hybrid union. **(3) Name fragmentation** (a player appears as Optimist Gerd, GERD, and Gerd_the_Optimist): a token-Jaccard resolver over case- and underscore-normalised tokens (3-char minimum) giving 100 percent csv coverage. **(4) Split accusation threads**: reply-chain expansion. **(5) Hallucinated CO claims** on translated text: grounding plus verbatim quote-validation. **(6) Non-deterministic retrieval**: a frozen per-game embedding cache. **(7) Idiomatic medium phrasings** ("the loincloth was black") evading patterns: a high-precision verdict-phrase set plus execution-anchoring (only verdicts about already-executed players are trusted).

1.3 Retrieval-Accuracy Optimisation Process

Retrieval was improved iteratively, each step measured on the 20-game validation set. **Step 1** pure embedding top-k: baseline, frequently dropped a player's own CO. **Step 2** add own posts and alias-aware mentions: recovered the decision-relevant subset that substring matching missed. **Step 3** add reply-chain expansion: preserved accusation and defence pairs, improving suspicion precision. **Step 4** add evidence-grounded prompting: prefixing regex-verified A2 facts cut fabricated claims and drove most of the plus 0.013 local rise from v4 to v6. **Step 5** add quote-validation feedback: deleted non-verbatim quotes so only retrieval-faithful signal reached the solver. The end state is a small, on-topic, verifiable context, the precondition for every later game-rule step.

1.4 Prompt Design and Optimisation Process

The prompt was also developed as a measured progression. **Step 1** a single general analyst prompt: brittle, it under-weighted procedural tells such as who consistently steered executions. **Step 2** two LLM calls averaged (v6): reduced variance but still one analytic lens. **Step 3** three deliberately-biased lenses whose opposite errors cancel under self-consistency (v7, plus 0.008 local, first Kaggle point 0.32205). **Step 4** add variant (c), a Madman-versus-Werewolf discriminator, because Madman is the metric's weakest class. **Step 5** defensive output handling: a JSON repair pass fixes truncated output, an omitted player falls back to the rule prior rather than zero, and every judgement must cite a verbatim quote so quote-validation can delete a fabricated claim without discarding the whole response. Each A3 call shares the identical contract in Table 1, runs at temperature 0 with a fixed seed, and three reruns are byte-identical.

Prompt part	Content given to Qwen-7B
System	Werewolf rules plus glossary (white is villager, black is wolf, CO is claim, GS is the white-to-black ranking, will is a death reveal)
Context	A2 evidence JSON (executions, night_deaths, co_seer, co_medium, co_hunter, divination_by) plus about 12 RAG posts
Task	JSON only: per player a wolf_score in 0 to 1 and role_scores over the 6 roles, each with a cited verbatim quote
Variant (a)	general behavioural analyst
Variant (b)	voting and accusation analyst: who drove each execution vote
Variant (c)	Madman-versus-Werewolf discriminator (weakest class, deliberately targeted)

Table 1. The A3 prompt skeleton; the three green rows are the self-consistency variants and share the identical System, Context and Task contract so their three outputs are directly averageable.

2. Optimisations and Improvements

Table 2 is the full Kaggle trajectory. A deterministic rule base (compute_v2_scores: night-death gives zero; executed gives a plus 0.15 prior; a single-CO Seer is boosted while multi-CO is damped; a Seer

black-call gives plus 0.20; a disclaim suppresses the special role) underlies the LLM, which only re-ranks within the freedom these priors leave, which is why a 7B model suffices. The late case-audit rows are Kaggle-only because they patch deterministic errors in the 30-game scored set, where a 20-game validation score is not meaningful.

Ver	Change	Local	Kaggle
v1	rule baseline (night-death 0, exec plus 0.15, seer-black plus 0.20)	0.219	n/a
v2	plus post evidence (CO, disclaim, divine results)	0.265	n/a
v4	plus Qwen-7B LLM, evidence-grounded	0.287	n/a
v6	plus RAG retrieval, two-LLM averaging	0.300	n/a
v7	plus 3 prompt-variant self-consistency	0.308	0.32205
v8-v13	exploratory LLM ensembles: reviewer / rank-agg / GLM-9B / 14B pairwise	0.30-.31	≤ v7, rejected
vC1_040	plus Medium-Execution chain (soft, conf-weighted)	0.322	0.34548
wolf_safe	flat max(ws,0.80) on 7 medium-conf wolves	n/a	0.33729 down
role_safe	ironclad Medium-fingerprint fix	n/a	0.34928 up
fingerprint	plus second Medium-fingerprint fix (private_14)	n/a	0.34973 up
nightdeath	ironclad night-kill is not Werewolf, FINAL	n/a	0.35026

Table 2. Trajectory 0.32205 to 0.35026 (the best Kaggle score among all submissions was about 0.543 at the time of submission, strong_baseline 0.21232). Blue: the pivotal gain; green: the final submission; red: a submitted regression kept as a documented lesson.

2.1 Key Engineering Wins

(i) **Constrained solver** fills roles greedily in fixed priority (Werewolf first, Villager last) under parsed counts, in order R times N, and decouples role from wolf_score (scored independently, so a row may be Villager with wolf_score 0.7); an early coupled version capped both metrics, and decoupling was about a plus 0.02 fix. (ii) **Split ensemble weights**: the LLM mainly affects wolf_score while rule-based role_scores are preserved for role assignment, which avoids contaminating Macro-F1 with noisy LLM role predictions while still improving Wolf-AP. (iii) **Medium-Execution chain (plus 0.022)**: each Medium claimant gets a confidence equal to the sigmoid of (score over 2), where score is plus 2 per executed target, minus 2 per living target, plus 1 if co-Medium, minus 1.5 if they disclaimed Medium; on the validation split the Medium-Execution chain raised Medium F1 from 0.25 to 0.40, while the confidence-weighted wolf-call nudge improved Wolf-AP without changing the role assignment. **Method-exhaustion discipline**: I pushed each idea until its marginal Kaggle return reached zero, then pivoted (tuning to game-rule chain, medium-wolf to ironclad role fixes, fingerprint to night-death), so the trajectory is a chain of exhausted, individually-verified directions, not a single lucky fit.

2.2 Deterministic Case-Audit Loop (final plus 0.0048)

After tuning saturated I mined the 30-game scored set for guaranteed errors using verifiable game rules only. The **Medium fingerprint** is an explicit Medium CO, sole claimant, no divination record, and reports only executed players; the **night-death rule** is that a wolf-killed player is never a Werewolf. A sweep of all 30 scored games confirmed these were the only ironclad violations; each patch is single-variable and individually Kaggle-verified (Table 3).

Step (single variable)	Kaggle	Delta	Verdict
vC1_040 (proven base, locked fallback)	0.34548	n/a	baseline
wolf_safe: flat max(ws,0.80) times 7	0.33729	minus 0.0082	reject
role_safe: Boy Peter Seer to Medium	0.34928	plus 0.0038	keep
fingerprint: plus Friedel (private_14)	0.34973	plus 0.00045	keep
nightdeath: Gerd Werewolf to Villager	0.35026	plus 0.00053	keep, final

Table 3. Case-audit ablation (red: submitted then rejected; green: the final). Example: in private_12, co_medium is Boy Peter alone, with no divination, reporting executed Jimzon and Nicholas, the exact fingerprint, yet the solver had him as Seer; the fix corrects a Medium false-negative, a Medium false-positive and a Seer false-positive in one game, giving plus 0.0038 on a 6-class Macro-F1.

3. Success and Failure Case Analysis

3.1 Headline, small-validation-set overfit. The 20-game validation set does not represent the 30-game scored set, so local gains systematically failed on Kaggle (v9h local plus 0.004 to Kaggle minus 0.011; v7fix plus 0.004 to minus 0.007; six-plus tuning attempts beat v7 locally and none on Kaggle). The `diagnose.py` confusion matrix showed the misclassified Seers were all in multi-CO games, the type over-represented in the small validation set, so fixing them is fitting noise. **3.2 Failure, wolf_safe** dropped 0.008: Wolf-AP is a rank metric and tying 7 rows at 0.80 destroyed the order vC1's soft nudge preserved. **3.3 Success, execution-anchored determinism:** every positive move has a regex-verifiable premise, and a 7B-plus-rules pipeline beating my own 14B and GLM-9B ensembles is the clearest evidence that capacity was never the bottleneck. **3.4** About twenty experiments were built and measured; Table 4 gives each one's root-cause analysis and the response strategy it triggered.

Attempt	Root-cause analysis	Response strategy
v8 reviewer agent	a second LLM critic shares the same translation-noise mode, so errors are correlated with no independent ground truth	drop LLM-on-LLM stacking; move signal to deterministic rules
v9 / v9h / v13 tuning	rank-blending and joint weight search fit the 20-game noise (Kaggle minus 0.011)	Kaggle-only single-variable tests; freeze weights
v10 14B, GLM-9B	larger and alternate models gave no gain; capacity was never the limit	drop to the compliant 7B model
v11 / v12 extractors	precision rose but special-role recall stayed too low to move the metric	accept the extraction-recall ceiling
v7fix multi-CO fix	local plus 0.004 but Kaggle minus 0.007 — the canonical overfit	founding evidence for the rules-only doctrine
graph features	vote-graph centrality is a behavioural-statistical signal, same overfit class	rejected under the doctrine
vB2 / vB3 ablations	removing the disclaim-zero or fake-role rules each hurt Kaggle	keep all v2 rules
vC2 seer-wolf nudge	a real Seer is indistinguishable in multi-CO fake-claim wars (zero fires)	use execution-anchored Medium, not Seer
vC4 Medium-human decay	human verdicts are noisier and AP only rewards pushing wolves up (asymmetric)	keep only the wolf direction of the chain
vC6 regex recall	widening verdict patterns dropped precision to 45 percent	precision-first, execution-anchored patterns only
vC7 / vD1-2 transitive	near-zero signal mass (one fire over 50 games), about zero expected value	do not spend submissions on marginal fires
vE1 3-judge extractor	42 candidates gave zero net-new clean wolf reveals — the true ceiling	accept the recall ceiling; use the case-audit
wolf_safe flat boost	saturating 7 rows destroyed Wolf-AP within-group order	only soft or single-cell changes thereafter

Table 4. Per-attempt root-cause analysis and the strategy each triggered. The recurring pattern, wins locally and loses on Kaggle, is exactly why the final system uses only verifiable game rules; each exhausted method directly motivated the next.

Final adopted analysis method. The procedure that emerged from this failure analysis, and which I finally adopted, is a strictly deterministic, execution-anchored decision method: (1) extract only regex-verifiable game events (executions, night-deaths, CO claims) as ground-truth anchors; (2) let the 7B LLM contribute behavioural suspicion only within the freedom the rule priors leave; (3) accept a downstream change only if its premise is a game rule and it yields a real, single-variable Kaggle gain against a locked fallback, reverting otherwise; (4) when an axis stops yielding, exhaust it and pivot to the next rule-anchored axis (the Medium-Execution chain, then the ironclad Medium fingerprint, then the night-death rule). This is precisely what converted a deceptive local signal into a monotone, individually-verified and reproducible improvement from 0.32205 to 0.35026. The key lesson is that high-precision deterministic evidence generalized, while statistical behavioural tuning overfit the 20-game validation set.

4. Results, Compliance and Reproducibility

The final Kaggle score is 0.35026 (strong_baseline 0.21232, simple_baseline 0.17349). **Compliance:** the final reproducible pipeline uses only Qwen2.5-7B-Instruct GGUF Q4, the assignment-recommended model, whose inference footprint is well under the 12 GB VRAM limit; there is no fine-tuning, no external API, and all inference is local. All discarded exploratory models (14B, GLM-9B) were used only for ablation, are not part of the submitted reproducible pipeline, and are not shipped. **Reproducibility:** the only stochastic stage, A3, is pinned to temperature 0 with a fixed seed and a frozen embedding cache; A3.5 and A5 are pure functions of regex-extracted game events; the patch step consumes fixed CSV inputs checked

by `validate_submission.py` (397 rows, 5 columns). Operationally I kept a locked fallback artifact (`vC1_040`, the last Kaggle-proven CSV): every later change was a single-variable submission accepted only on a real Kaggle gain and otherwise reverted, so the final CSV is a monotone chain of individually-verified rule-grounded edits; the final score stayed stable because each accepted change was a single-variable, rule-grounded Kaggle improvement, not a local tuning gain. The README lists the exact commands; the key entry point is `python main.py` with flags `--medium-wolf-nudge 0.40 --chain-scale 0.10 --no-multico-fix` after the single Qwen-7B step, followed by the deterministic case-audit. Table 5 summarises the three steps.

Step	Action	Uses LLM	Produces
1 <code>run_llm.py</code>	A2 evidence plus RAG plus 3-variant Qwen-7B reasoning (temp 0)	Yes, 7B only, under 12 GB	LLM wolf and role JSON
2 <code>main.py</code>	constrained solver plus Medium-Execution chain	No	<code>vC1_040</code> base CSV
3 <code>apply_patch.py</code>	ironclad case-audit plus <code>validate_submission</code> format check	No	FINAL CSV, 397 by 5

Table 5. The three-step reproduction (full commands in the README); the gold row is the only model call (Qwen-7B, under 12 GB) and steps 2 and 3 are deterministic. A pre-computed final CSV is also shipped for direct verification. In sum, a disciplined preference for verifiable game-rule signals over statistical tuning produced a robust, interpretable, compliant system that moved the score from a 0.21232 baseline to 0.35026 within a reproducible under-12-GB Qwen-7B pipeline.

Future work. The attainable ceiling is bounded by evidence-extraction recall, not downstream modelling: many true Medium and Seer reveals are never cleanly extracted from the noisy translated logs, and the vE1 experiment showed recall cannot be lifted by prompt engineering alone. The highest-value next step is a higher-recall but still execution-anchored extraction stage, for example a per-execution windowed LLM pass that reconstructs the full Medium-reveal sequence under the same quote-validation discipline; I did not attempt this in the final system because preliminary vE1 results showed that looser extraction reduces precision and hurts Wolf-AP. A second avenue is modelling the Madman from voting-pattern anomalies rather than CO contradictions, since true Madmen almost never claim a role. Both directions keep the central discipline of this project: invest effort in recall of verifiable game evidence, not in tuning a small and unrepresentative validation set.